

2026-06-19 Eval-процесс разработки ЦПТ

Eval-процесс разработки ЦПТ

Зачем нужен eval-процесс

Главный вопрос: как разработчик при отладке может понять, что ЦПТ начал выдавать лучший результат?

Ответ: нужен не разовый чек-лист, а постоянный eval-процесс. Он должен работать как регрессионное тестирование для LLM-функциональности.

Базовая схема:

эталонный датасет → прогон ЦПТ → автоматическая оценка → сравнение с предыдущим прогоном → анализ деградаций → доработка → новый прогон

Без такого процесса качество будет оцениваться субъективно: “кажется, стало лучше”. Это опасно, потому что можно улучшить один пример и сломать несколько других.

Что такое eval harness

Eval harness — это инструмент, который позволяет:

- запускать ЦПТ на фиксированном наборе примеров;
- сохранять результат каждого прогона;
- считать метрики;
- сравнивать результат с предыдущими версиями;
- показывать, какие кейсы улучшились, а какие деградировали.

Это не обязательно сложная система на первом этапе. На старте это может быть набор скриптов, таблиц и ручной экспертной проверки. Но логика должна быть именно такой: фиксированный датасет, повторяемый прогон, сравнимые метрики.

Датасеты

Нужно три уровня датасетов.

Smoke dataset

Небольшой набор на 5–10 кейсов.

Назначение:

- быстро проверить, что базовые сценарии не сломались;
- использовать после каждой правки промпта, кода, RAG, чанкинга, reranker или схемы ответа;
- запускать локально или в контуре разработки.

Пример использования:

разработчик поменял промпт → запустил smoke eval → увидел, что базовые кейсы не сломались

Smoke dataset не должен быть большим. Его цель — скорость обратной связи.

Regression dataset

Основной набор на 50–150 кейсов.

Назначение:

- проверять качество перед merge;
- проверять качество перед демо;
- проверять качество перед поставкой;
- отслеживать регрессии.

Каждая найденная ошибка должна добавляться в regression dataset. Это ключевой механизм накопления качества.

Пример:

нашли, что ЦПТ придумывает endpoint в API-кейсе → добавили такой пример в regression dataset → в следующих прогонах проверяем, что ошибка не вернулась

Corner cases dataset

Набор сложных случаев.

В него должны входить:

- плохие формулировки требований;
- неполные требования;
- требования со статусом `insufficient_detail`;
- большие документы;
- несколько источников;

- дублирующиеся требования из разных источников;
- смешанные API / WEB / интеграционные сценарии;
- требования с неоднозначной классификацией;
- случаи, где ЦПТ склонен галлюцинировать.

Corner cases не обязательно гонять после каждой правки. Их можно запускать nightly или перед релизом.

Что должно быть в эталонном датасете

Каждый пример в датасете должен содержать не только исходный документ, но и эталонный результат.

Минимальный состав:

1. Исходный источник.
2. Эталонный список атомарных требований.
3. Статусы требований.
4. Эталонные ТК.
5. Связи `требование` → `эталонные ТК`.
6. Ожидаемый вид ТК.
7. Ожидаемое направление ТК.
8. Список обязательных проверок.

Без эталона невозможно объективно оценивать качество.

Что сохранять в каждом прогоне

Каждый прогон должен иметь `generationRunId`.

Нужно сохранять:

- дату и время запуска;
- версию кода;
- версию промпта;
- версию модели;
- версию датасета;
- параметры RAG / чанкинга / reranker;
- результат выделения требований;
- результат генерации ТК;
- связи `ТК` → `требования`;
- рассчитанные метрики;

- список ошибок.

Это нужно, чтобы понимать, какое именно изменение повлияло на качество.

Метрики

Метрики нужно считать по блокам, а не одним общим числом.

Метрики декомпозиции

Метрика	Что показывает
Requirement extraction recall	сколько эталонных требований ЦПТ нашел
Requirement extraction precision	сколько выделенных ЦПТ требований действительно корректны
Boundary accuracy	насколько корректны границы требований
Consolidation accuracy	насколько корректно объединяются требования из разных источников
Insufficient detail accuracy	насколько корректно ЦПТ определяет insufficient_detail

Метрики генерации

Метрика	Что показывает
Requirement coverage	доля пригодных требований, корректно покрытых ТК
Positive coverage	доля требований с позитивным ТК
Negative coverage	доля требований с негативным ТК
Etalon match	соответствие эталонным ТК по смыслу и структуре
Extra relevant TC rate	доля дополнительных ТК, которые релевантны и не являются дублями

Метрики структуры

Метрика	Что показывает
Schema pass rate	доля ТК, соответствующих структуре ответа
Required fields completeness	доля ТК с заполненными обязательными полями

Метрика	Что показывает
Expected result completeness	доля шагов с ожидаемым результатом
Step atomicity rate	доля шагов, где один шаг — одно действие

Метрики классификации

Метрика	Что показывает
Test case type accuracy	корректность вида ТК
Direction accuracy	корректность направления позитивный / негативный

Метрики качества

Метрика	Что показывает
Hallucination count	количество ТК с галлюцинациями
Unsupported specificity count	количество случаев необоснованной конкретизации
Irrelevant TC count	количество нерелевантных ТК
Broken expected result count	количество ОР, не соответствующих шагу

Метрики производительности

Метрика	Что показывает
Average generation time	среднее время генерации
P95 generation time	P95 времени генерации
Time by stage	время по этапам: загрузка, парсинг, embeddings, RAG, LLM, постобработка

Как видеть прогресс

Разработчик должен видеть отчет “было / стало”.

Пример:

Метрика	Было	Стало	Изменение
Полнота выделения требований	82%	89%	+7 п.п.

Метрика	Было	Стало	Изменение
Корректность границ	76%	84%	+8 п.п.
Покрытие требований ТК	68%	81%	+13 п.п.
Классификация вида ТК	91%	93%	+2 п.п.
Галлюцинации	7	3	лучше
Schema pass rate	100%	100%	без изменений
Среднее время генерации	5:20	6:10	хуже

Такой отчет показывает не только качество, но и trade-off. Например, качество могло улучшиться, но производительность ухудшилась.

Diff по кейсам

Агрегированных метрик недостаточно. Нужен diff по конкретным кейсам.

Пример:

Dataset case	Было	Стало	Комментарий
DOC-001	failed	passed	начал правильно выделять требования
DOC-002	passed	failed	деградация: потерял негативный ТК
DOC-003	failed	failed	все еще галлюцинирует endpoint
DOC-004	partial	passed	улучшилось покрытие

Внутри каждого кейса нужно показывать изменения по требованиям:

REQ-001

Было: нет негативного ТК

Стало: негативный ТК появился

Статус: улучшение

REQ-002

Было: корректно

Стало: появилась галлюцинация “кнопка Сохранить”

Статус: деградация

Hard gates

Для merge / поставки нужны блокирующие правила.

Пример hard gates:

- schema pass rate = 100%;
- покрытие эталонного ТЗ 30.06 = 100%;
- нет галлюцинаций на эталонном ТЗ;
- не ухудшилось покрытие требований относительно предыдущей версии;
- не выросло количество галлюцинаций;
- не сломались кейсы из smoke dataset.

Процесс для команды

Для разработчика

изменил промпт / RAG / код → запустил smoke eval → посмотрел diff → исправил
→ повторил

Для merge request

прогон regression dataset → сравнение с baseline → проверка hard gates →
merge или возврат на доработку

Для релиза

прогон regression + corner cases → проверка порогов → ручная экспертная
проверка спорных кейсов → решение о поставке

Вывод

Разработчику нужен не чек-лист, а инструментальная петля качества. Прогресс должен быть виден через:

- стабильный датасет;
- повторяемый прогон;
- метрики;
- diff по кейсам;
- историю прогонов;
- регрессионный набор ошибок.

Только так можно управляемо улучшать ЦПТ.